



ERAASOFT

Task 12

Osama Sabry Fekry

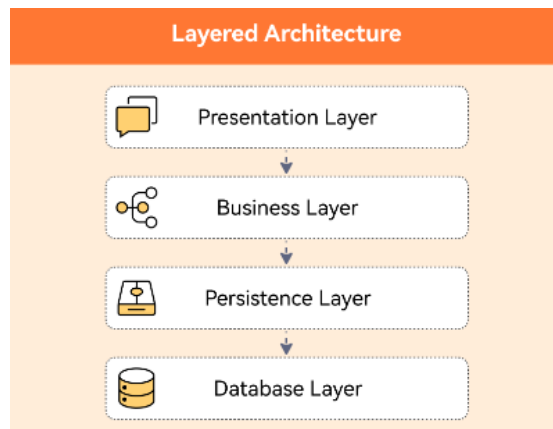
Architecture Patterns:-

It is a high-level way to organize a complete software system. It describes the main parts, their responsibilities and how they communicate.

Types of Architecture Patterns

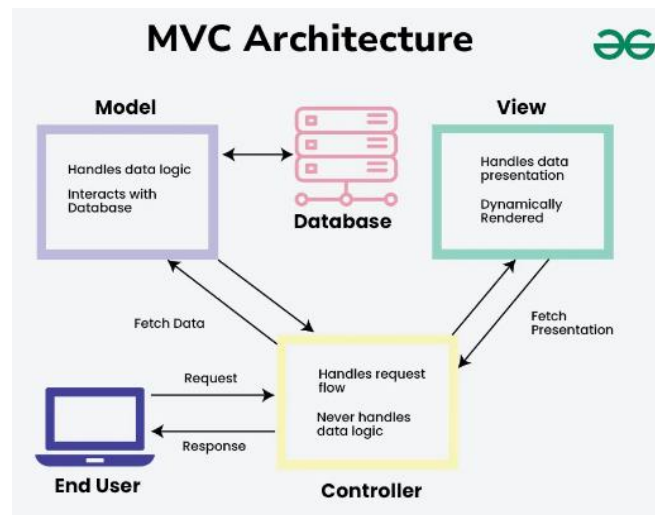
Layered Architecture

Layered Architecture separates the application by responsibility. Common layers are presentation, business logic and data access. A layer normally uses the layer below it, so changes can be contained and each part is easier to test. It works well for many business applications, but too many layers can make a small change pass through unnecessary code.



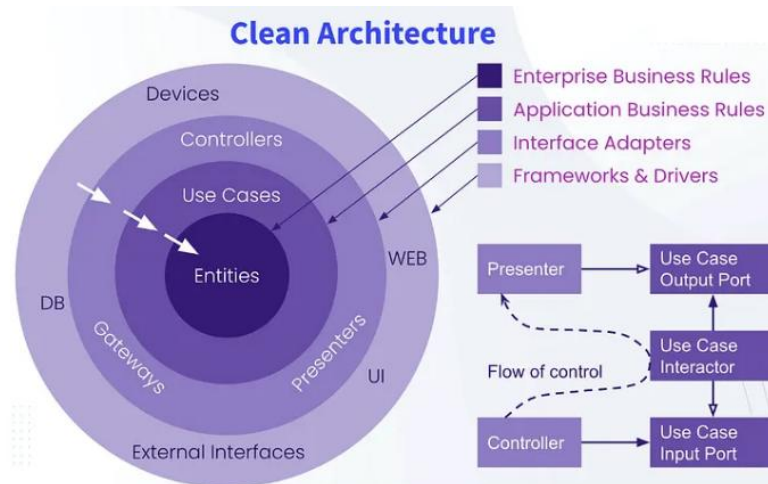
MVC (Model-View-Controller)

MVC separates user-interface work into the Model, View and Controller. The Model holds data and rules, the View displays information, and the Controller handles requests and selects the next action. This makes interface code easier to change and test. MVC can be used inside a monolithic, layered or service-based system.



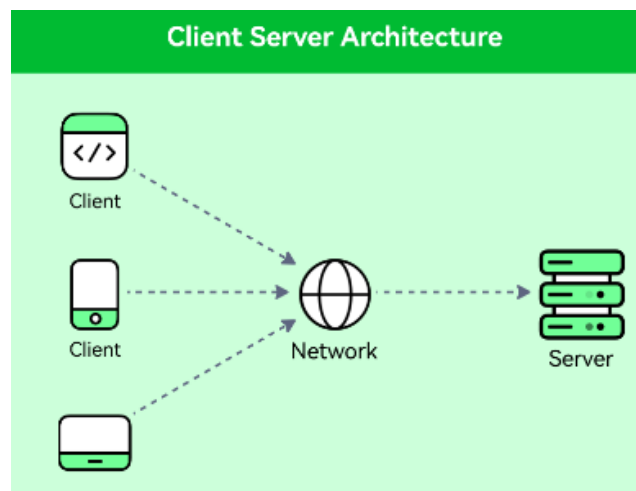
Clean Architecture

Clean Architecture keeps business rules in the center and makes outer tools depend on them through interfaces. The database, web framework and user interface become replaceable details. This improves testing and protects important rules from technology changes, but it adds interfaces and structure that may be unnecessary for a very small program.



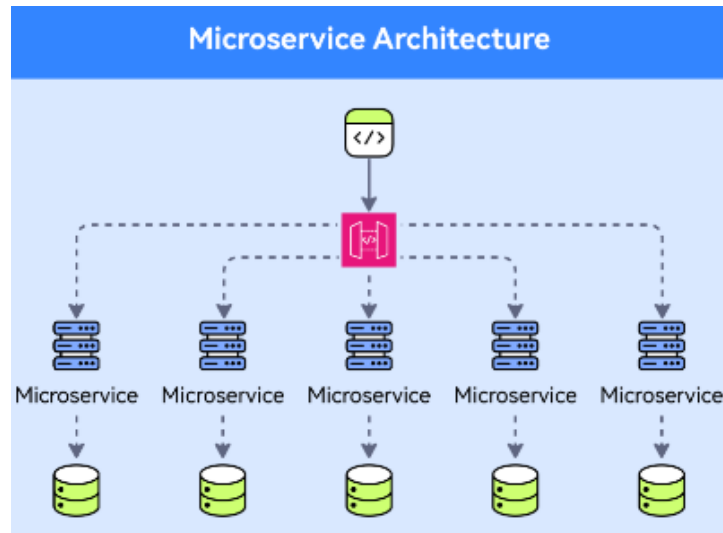
Client-Server

Client-Server divides a networked system into clients that request services and servers that process requests and manage shared resources. It allows many users to use the same data or service. The design must also handle network delays, security, server capacity and what happens when the connection fails.



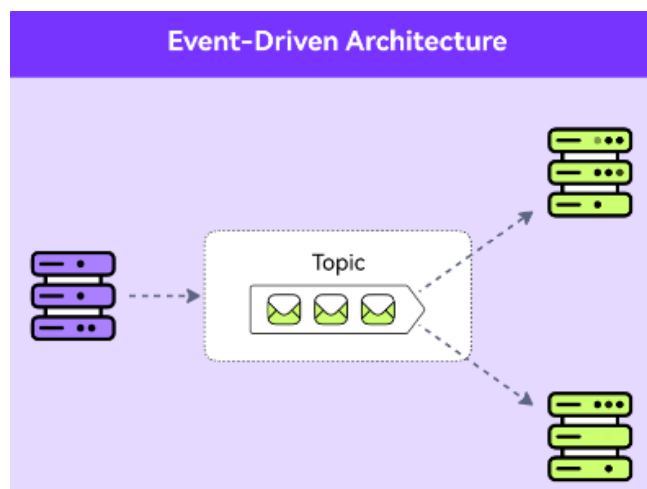
Microservices

Microservices divide a large application into small services built around business capabilities. Each service can be developed, deployed and scaled independently. This supports large teams and uneven workloads, but it creates extra work for network communication, monitoring, testing and keeping data consistent.



Event-Driven Architecture

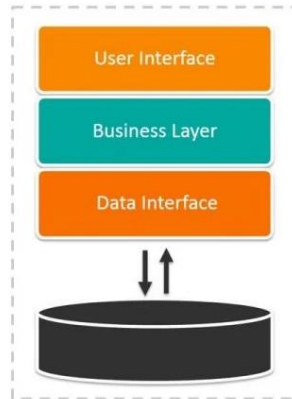
In Event-Driven Architecture, a producer publishes an event when something happens and one or more consumers react to it. A message broker is often placed between them. This reduces direct dependency and supports background processing, but event order, duplicate messages and eventual consistency must be managed carefully.



Monolithic Architecture

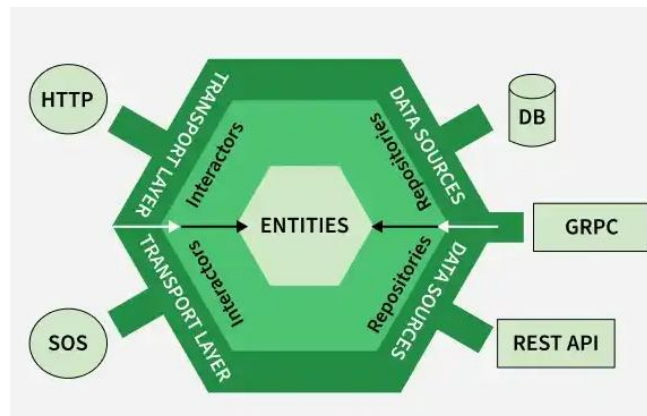
A monolithic application is built and deployed as one unit, even when its internal code is divided into clear modules or layers. It is usually simple to start, test and deploy, so it is a practical choice for small systems and early products. As it grows, one change can require a full release and the whole application may need to scale even when only one feature is busy.

Monolithic Architecture



Hexagonal Architecture

also called Ports and Adapters, places the business rules in the center. The core uses ports, which are interfaces that say what it needs, while adapters connect those ports to a database, user interface, API or message queue. This makes the core easier to test and lets a team replace an outside technology without changing the main business rules. The extra interfaces may be too much for a very small program.



These patterns answer different questions. Layered, MVC, Clean and Hexagonal Architecture mainly organize responsibilities and dependencies. Client-Server explains how software is divided across a network. Monolithic and Microservices mainly describe deployment boundaries, while Event-Driven Architecture describes communication. They are not always alternatives. For example, an online shop can use MVC and layers inside a monolith or use Hexagonal Architecture inside each microservice and connect the services with events.

Design patterns

It is a smaller reusable idea for a repeated problem inside the code, such as creating an object or changing its behavior.

Types of Design Patterns

Creational patterns

Creational patterns provide flexible ways to make objects and reuse creation code. Its five patterns are Factory Method, Abstract Factory, Builder, Prototype and Singleton.

Factory (Factory Method)

Factory Method provides a creation method but lets subclasses decide which concrete object will be returned. It is helpful when the main code should work with a common type and should not be tied to the exact class it creates.

Abstract Factory

Abstract Factory creates families of related objects through common interfaces. It helps an application produce matching products, such as buttons and menus for one operating system, without naming every concrete class in the main code.

Builder

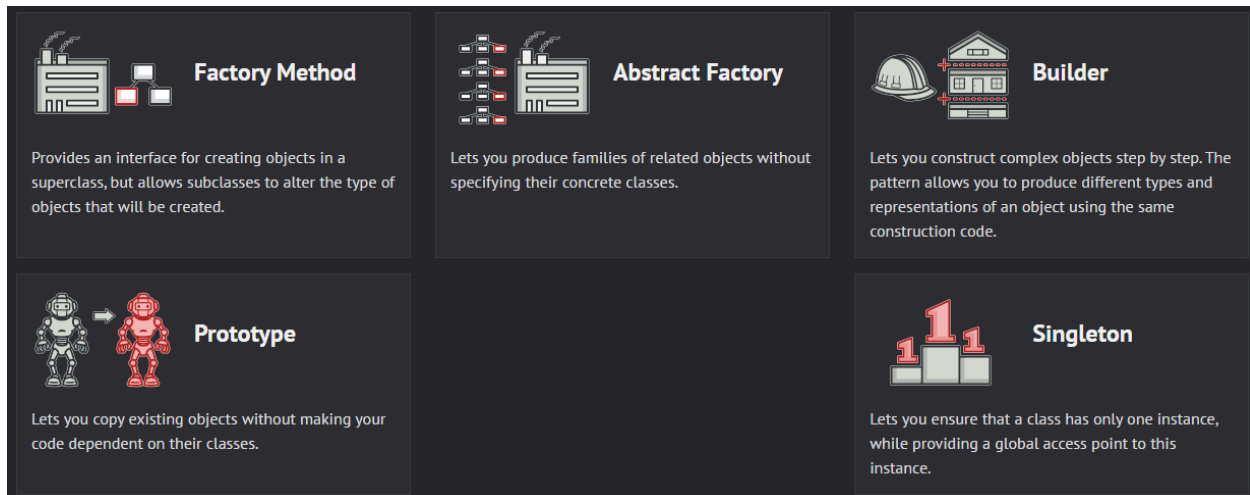
Builder constructs a complex object through clear steps. The same building process can create different versions, so a large constructor does not need many optional parameters.

Prototype

Prototype creates a new object by copying an existing object. It is useful when an object has an expensive or complicated setup that should not be repeated from the beginning.

Singleton

Singleton ensures that a class has one shared instance and provides a common way to reach it. It can be useful for a truly shared service, but too much global access can make testing and dependencies harder to manage.



Structural patterns

Structural patterns show how classes and objects can be joined into larger, flexible structures. Its seven patterns are Adapter, Bridge, Composite, Decorator, Facade, Flyweight and Proxy.

Adapter

Adapter translates one interface into another interface that the client expects. It allows new code to use an old class or outside library without changing either side's main logic.

Bridge

Bridge separates an abstraction from its implementation so both can change on their own. This avoids creating many subclasses for every possible combination of a feature and a platform.

Composite

Composite places individual objects and groups in a tree and lets the client treat both in the same way. It fits part-whole structures such as folders, menus or company departments.

Decorator

Decorator adds behavior by placing an object inside a wrapper with the same interface. Several decorators can be combined, making features more flexible than a large set of subclasses.

Facade

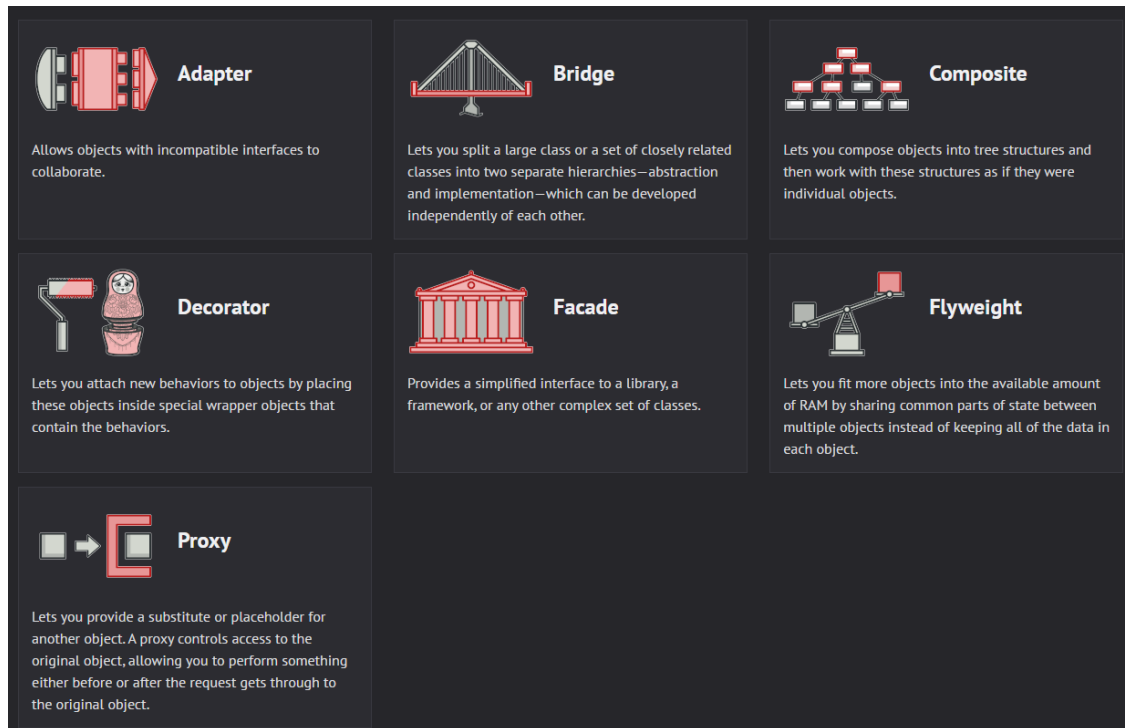
Facade gives clients one simple entry point to a complex library or subsystem. It reduces the number of internal classes that the client must understand and use directly.

Flyweight

Flyweight saves memory by sharing data that is the same across many small objects. Each object keeps only its unique data, while common details are stored once and reused.

Proxy

Proxy is a substitute that uses the same interface as the real object and controls access to it. The proxy can check permission, delay loading, cache a result or record the request before passing it on.



Behavioral patterns

Behavioral patterns focus on algorithms, responsibilities and communication between objects. Its ten patterns are Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method and Visitor.

Chain of Responsibility

Chain of Responsibility passes a request through a sequence of handlers. Each handler either processes the request or sends it to the next handler, so the sender does not need to know which handler will finish the work.

Command

Command turns a request into an object that stores the action and its information. This makes an action easier to pass around, queue, record or undo.

Iterator

Iterator gives a standard way to move through a collection without showing how the collection is stored. The same client code can therefore work with lists, trees or other collection types.

Mediator

Mediator places communication rules in one central object instead of allowing many objects to call each other directly. It reduces tangled dependencies when several components must coordinate.

Memento

Memento saves a snapshot of an object's state so that the state can be restored later without exposing private details. It is often used for undo, history or checkpoints.

Observer

Observer lets objects subscribe to updates from another object. When the subject changes, it notifies all subscribers without being tightly connected to their specific classes.

State

State moves the behavior for each condition into a separate state object. The main object changes its behavior by changing its current state instead of using one large set of conditions.

Strategy

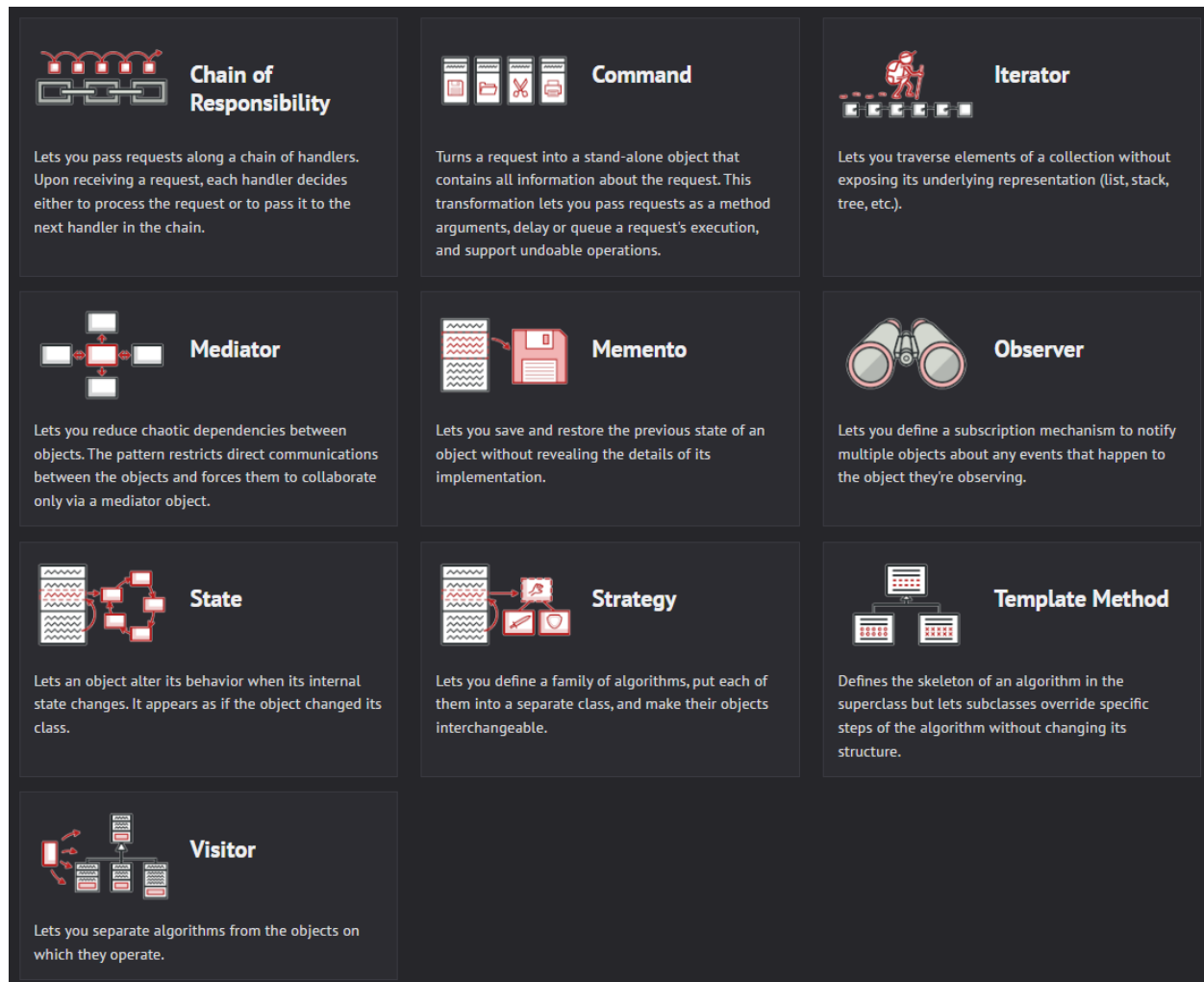
Strategy puts different ways of completing a task into interchangeable objects. A program can select or replace the algorithm while the code using it stays the same.

Template Method

Template Method defines the fixed order of an algorithm in a base class and lets subclasses change selected steps. It reuses the common workflow while still allowing some variation.

Visitor

Visitors place an operation in a separate visitor object instead of adding it to all the classes it works on. It is useful when many different operations must run across a stable group of object types.



No pattern is best for every project. A good choice matches the size of the system, the expected changes, the team's skills and the cost of extra complexity.

Summary for all patterns

Architecture patterns

Layered Architecture

What: Splits an application into layers, such as presentation, business rules and data access.

Problem: Keeps different jobs from becoming mixed together.

Use: When an application needs a clear structure that is easy to maintain.

Example: A school app has a screen, a grading service and a layer that reads marks.

MVC (Model-View-Controller)

What: The Model manages data and rules, the View shows information, and the Controller handles user actions.

Problem: Separates the screen from the work behind it.

Use: In applications with a user interface, including websites.

Example: In a shop website, the controller handles a product request and the view displays product data from the model.

Clean Architecture

What: Keeps business rules at the center. Code dependencies point inward toward those rules.

Problem: Stops core rules depending on a particular database, screen or framework.

Use: In systems that need easy testing and may change their tools later.

Example: A booking system can replace its database while keeping the same booking rules.

Client-Server

What: A client asks for a service, and a server processes the request and sends a response.

Problem: Lets many users access shared data and services.

Use: When devices need to communicate with a central service over a network.

Example: A browser asks a web server for a page, and the server sends it back.

Microservices

What: Splits a system into small services that communicate and can be deployed separately.

Problem: Lets teams update or grow one service without releasing the whole system.

Use: In large systems where different features need independent development or scaling.

Example: An online shop has separate services for products, payments and delivery.

Event-Driven Architecture

What: Parts of a system react to events, which are messages saying something happened.

Problem: Lets several parts respond without the sender calling each one directly.

Use: When actions should trigger follow-up work, often in the background.

Example: An **OrderPlaced event** starts stock updates and a confirmation message.

Monolithic Architecture

What: Builds and deploys the main application as one unit. It can still contain organized layers.

Problem: Keeps development and deployment simpler by avoiding many separate services.

Use: For a small application or an early version of a product.

Example: A small shop manages products, orders and users in one application.

Hexagonal Architecture (Ports and Adapters)

What: Keeps business rules in the center and connects outside systems through ports and adapters.

Problem: Stops databases, screens and external services from controlling the core logic.

Use: When a system must be easy to test or may replace outside technologies.

Example: A shop connects its payment rules to either a PayPal or Stripe adapter.

Creational design patterns

These patterns deal with creating objects.

Factory (Factory Method)

What: Uses a creation method while subclasses choose the concrete object it returns.

Problem: Reduces dependence on specific classes.

Use: When the type of object may change but the client uses one common interface.

Example: A notification creator makes either an **EmailNotification** or **SMSNotification**.

Abstract Factory

What: Creates a family of related objects through common interfaces.

Problem: Keeps matching products together without naming their concrete classes.

Use: When an application supports different platforms, themes or product families.

Example: A Windows factory creates matching Windows buttons and menus.

Builder

What: Builds a complex object step by step.

Problem: Simplifies setup with many options.

Use: When objects need different combinations of settings.

Example: Building a computer order with selected memory, storage and processor.

Prototype

What: Creates an object by copying an existing one.

Problem: Avoids repeating object setup.

Use: When many objects start with similar settings.

Example: Copying a game enemy and changing its name.

Singleton

What: Keeps one shared instance of a class and provides a common access point.

Problem: Avoids separate copies of something that should be shared.

Use: When one shared object is truly needed in the running application.

Example: Several parts of an app use the same **settings manager**.

Structural design patterns

These patterns deal with connecting or wrapping objects.

Adapter

What: Changes an object's interface into one another part expects.

Problem: Connects incompatible interfaces.

Use: When new code must work with an older library.

Example: Wrapping an old payment service behind a new `pay()` method.

Bridge

What: Separates an abstraction from its implementation so each can change independently.

Problem: Avoids making a subclass for every combination of feature and platform.

Use: When two parts of a design can vary in separate ways.

Example: Remote controls work with different television brands through a device interface.

Composite

What: Places single objects and groups in a tree and treats them in the same way.

Problem: Simplifies work with part-whole structures.

Use: When data naturally forms a tree.

Example: Files and folders can both report their size.

Decorator

What: Wraps an object to add behavior while keeping the same interface.

Problem: Adds features without changing the original class.

Use: When features should be combined flexibly.

Example: Adding milk and sugar wrappers to a coffee order.

Facade

What: Gives a simple interface to a group of more complex parts.

Problem: Hides the many steps needed to use a subsystem.

Use: When callers should not need to know the internal details.

Example: A `start_movie()` method turns on the screen, speakers and player.

Flyweight

What: Shares common data between many small objects to save memory.

Problem: Avoids storing the same information inside thousands of objects.

Use: When a program creates a very large number of similar objects.

Example: A map shares one tree image and color across thousands of tree positions.

Proxy

What: Stands in for another object and controls access through the same interface.

Problem: Adds access checks or delays expensive work until needed.

Use: When access must be controlled or loading is costly.

Example: An image proxy loads the full picture only when it is displayed.

Behavioral design patterns

These patterns deal with how objects act and communicate.

Chain of Responsibility

What: Passes a request through a chain of possible handlers.

Problem: Keeps the sender from depending on one specific handler.

Use: When several handlers may check or process a request in order.

Example: A support request moves from a chatbot to an agent and then a manager.

Command

What: Stores an action and its needed information in an object.

Problem: Makes actions easier to queue, record or undo when undo logic is provided.

Use: In editors, job queues and button actions.

Example: A text editor stores a typing command so the user can undo it.

Iterator

What: Moves through a collection without exposing how the collection is stored.

Problem: Gives clients one simple way to visit different collection types.

Use: When code must read items from lists, trees or other collections.

Example: A music playlist iterator returns one song at a time.

Mediator

What: Uses a central object to manage communication between several objects.

Problem: Reduces confusing direct connections between components.

Use: When many interface controls or services must coordinate.

Example: An airport control tower coordinates planes instead of planes calling each other.

Memento

What: Saves an object's state so it can be restored later without exposing private details.

Problem: Supports undo or history without breaking the object's protection.

Use: When users may need to return to an earlier state.

Example: A drawing app saves a snapshot before an edit and restores it for Undo.

Observer

What: Lets an object notify its subscribers when something changes.

Problem: Keeps interested objects updated without repeated checking.

Use: For notifications or screens that react to changing data.

Example: A weather station notifies several displays when the temperature changes.

State

What: Puts behavior for each condition in a separate state object.

Problem: Reduces large sets of conditions for different states.

Use: When the same action should behave differently depending on the current state.

Example: A vending machine handles a selection differently before and after payment.

Strategy

What: Keeps different ways of doing a job in separate, interchangeable objects.

Problem: Allows behavior to change without rewriting the class that uses it.

Use: When a task has several possible methods.

Example: An order uses either standard or express delivery to calculate its total.

Template Method

What: Defines an algorithm's main order while subclasses change selected steps.

Problem: Reuses a common process without copying its whole sequence.

Use: When several tasks follow the same workflow but differ in a few steps.

Example: Report classes load data format it differently and then saves it.

Visitor

What: Keeps an operation separate from the object types on which it works.

Problem: Adds new operations without changing every class into a stable object structure.

Use: When many operations must work across the same group of object types.

Example: A shopping-cart visitor calculates tax differently for books and food.

Patterns that I chose

I chose MVC because it separates the data, user interface, and user actions, making the code easier to organize and maintain. I chose Builder because it creates complex objects through simple steps and supports different options. I chose Observer because it automatically informs multiple objects when a change happens, without connecting them directly.

MVC Architecture: student results

StudentModel stores the marks and decides whether each student passes or fails. **StudentView** displays the result. **StudentController** receives a student's name, asks the Model for the result and sends that result to **the View**. This keeps the three MVC responsibilities separate and makes the program easier to change.

Output:

Osama: 85/100 - Pass

Ali: 40/100 - Fail

Builder Pattern: computer order

ComputerBuilder creates a Computer in small steps. The **add_cpu()**, **add_memory()** and **add_storage()** methods return the same builder, so the steps can be joined in one clear chain. The **build()** method returns the finished Computer object. This is useful when an object has several optional parts or settings.

Output:

Computer order:

CPU: Intel Core i5

Memory: 16 GB

Storage: 512 GB SSD

Observer Pattern: weather updates

WeatherStation keeps a list of observers. **PhoneDisplay** and **WindowDisplay** are added to that list. When the temperature changes, the station calls **update()** on every observer, so both displays receive the new value automatically. The station does not need to know how each display uses the information.

Output:

Phone display: 30 C

Window display: 30 C